

TILEGRAD 1.0 Language Specification

A tile programming language over TINYGRAD UOps

TileGrad contributors

July 2026

Purpose

TILEGRAD is a Python-embedded language for explicitly scheduled tiled kernels. It provides TileLang-like ergonomics while using TINYGRAD UOps as the sole normative backend IR for TILEGRAD 1.0.

TILEGRAD owns tile-level semantics, storage placement, scheduling intent, validation, and inspection. TINYGRAD owns scalar semantics, instruction selection, register and memory planning, rendering, compilation, and execution.

TileGrad is intended for fused and specialized tinygrad operators such as tiled GEMM, attention, normalization, quantized kernels, and research dataflows that require explicit scheduling beyond the Tensor API.

The central compilation path is:

TileGrad source $\rightarrow$ Tile IR $\rightarrow$ scheduled Tile IR $\rightarrow$ tinygrad UOp Sink $\rightarrow$ tinygrad program

This document specifies a stable Core language and optional Standard and Target profiles. A Core operation has a portable scalar or SIMT lowering. Profiles add native matrix operations, software pipelines, atomics, subgroup collectives, and target-specific mechanisms when the pinned TINYGRAD integration can represent them safely.

Portable Core does not depend on a vendor, renderer family, native matrix facility, subgroup width, or Target profile. For every target on which the pinned TINYGRAD integration can legally lower and execute the Core UOps required by a program, TILEGRAD **MUST** provide the Core lowering without a second backend.

On capability tuples with suitable typed native matrix support, the Native Matrix profile targets TileLang-like performance for ordinary dense matrix workloads. Performance is a target-scoped, versioned conformance claim, not a promise of exact or universal TileLang parity.

The keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY** indicate normative requirements.

Ownership Boundary

TILEGRAD **MUST** provide:

- one canonical Tile IR shared by the parsed and builder frontends;
- explicit execution axes, storage scopes, tile views, layouts, and tile operations;
- validation before constructing TINYGRAD UOps;
- a schedule-preserving adapter to a pinned TINYGRAD revision range;
- inspection of every Tile IR stage and the emitted UOp graph.

TILEGRAD **MUST NOT** provide a second scalar ALU, renderer, compiler, runtime, or numerical contract. It **MUST NOT** promise a target mechanism that the pinned TINYGRAD integration cannot represent with typed operations and launch metadata.

TILEGRAD may select a capability class and the schedule and outer layouts required by that class. The pinned TINYGRAD integration owns the final native instruction configuration, atom-level lane mapping, rendering, compilation, and execution within the validated class.

Programs and Frontends

A program is a Python function decorated with `@tg.kernel`. The decorator parses a restricted Python subset and returns a callable `TILEKERNEL`. The public `KernelBuilder` API constructs the same IR directly; it is not a second language.

`TILEKERNEL` is a frontend compiler and specialized, not a runtime JIT. On invocation it binds compile-time and shape-specialized values, selects a cached specialization or creates one, allocates declared outputs, and constructs a `Tensor.custom_kernel` call.

```
@tg.kernel
def matmul(A, B, *, BM: T.Const[int] = 64,
           BN: T.Const[int] = 64, BK: T.Const[int] = 32):
    M, N, K = T.const("M", "N", "K")
    A: T.Tensor[(M, K), T.float16]
    B: T.Tensor[(K, N), T.float16]
    C = T.empty((M, N), T.float16)

    with T.Kernel(T.ceildiv(N, BN), T.ceildiv(M, BM),
                  threads=128) as (bx, by):
        As = T.shared((BM, BK), T.float16)
        Bs = T.shared((BK, BN), T.float16)
        acc = T.fragment((BM, BN), T.float32)
        T.clear(acc)
        for ko in T.Serial(T.ceildiv(K, BK)):
            T.copy(A.tile((by*BM, ko*BK), (BM, BK)), As, pad=0)
            T.copy(B.tile((ko*BK, bx*BN), (BK, BN)), Bs, pad=0)
            T.mma(As, Bs, acc)
        T.copy(acc, C.tile((by*BM, bx*BN), (BM, BN)))
    return C
```

Specialization

Values are classified as follows:

Class	Meaning	Legal use
Compile-time	Python value included in the specialization key.	Shapes, layouts, launch geometry, unrolling, and scheduling.
Shape-specialized	Value bound from a tensor shape or stride and included in the key.	Same uses as a compile-time value after binding.
Runtime	Device scalar not specialized by value.	Scalar expressions, predicates, indices, and serial loop bounds.

Runtime values **MUST NOT** determine allocation or launch shapes. The specialization key **MUST** include all values affecting generated UOps, including dtype, rank, shape, admitted strides, target, layouts, numerical mode, selected profile names and versions, advertised capability tuple, tuned schedule configuration, TileGrad revision, and pinned TINYGRAD revision.

Repeated symbols in tensor and shape declarations **MUST** bind to the same value. Outputs **MUST NOT** alias an input or another output unless the declaration explicitly permits the alias and validation proves the effects legal.

TileGrad owns only the specialization cache from this key to scheduled Tile IR and its UOp Sink. TINYGRAD owns program compilation and binary caches.

Kernel regions are submitted in source order. An implementation may overlap independent regions only when TINYGRAD represents the required dependencies and the overlap preserves every data and effect ordering. Returning a tensor does not require a host synchronization. Workgroups in one launch are unordered.

tinygrad composition

A `TILEKERNEL` **MUST** compose inside TINYGRAD `@function(precompile=True)` and inside an enclosing `TinyJit`. `@function` provides a reusable parameterized Tensor graph and optional precompilation; `TinyJit` captures and re-

plays an enclosing execution graph. TileGrad **MUST NOT** automatically create a `TinyJit`, because JIT ownership belongs to the caller and nested `TinyJit` capture is unsupported.

Types and Scalar Expressions

TILEGRAD inherits scalar dtypes, constants, promotion, casts, bitcasts, and ALU semantics from the pinned TINYGRAD revision. Core includes booleans, signed and unsigned integers, `float16`, `bfloat16`, `float32`, and `float64`. A target execution claim applies only when the pinned TINYGRAD target can legally lower the dtypes used by the program. Unsupported target and dtype combinations **MUST** be diagnosed rather than silently substituted. FP8, sub-byte, packed, and mixed low-precision formats require an advertised Standard or Target profile.

Scalar and index expressions lower directly to TINYGRAD UOps. Tile IR **MUST NOT** define an independent arithmetic graph. The frontend may wrap UOps to preserve source locations and TileGrad type diagnostics.

Index arithmetic **MUST** be integral and **MUST NOT** wrap to produce a valid address. Floating-point and integer results follow TINYGRAD semantics.

A versioned numerical conformance manifest **MUST** record the selected TINYGRAD numerical mode and the permitted rounding, reassociation, contraction, approximation, denormal, NaN, infinity, signed-zero, and tolerance behavior for every tested operation and profile. The manifest makes inherited TINYGRAD semantics reproducible; it does not define a second numerical contract.

Buffers, Views, and Storage

A buffer has a dtype, shape, storage scope, access mode, and optional layout. A view is $(B, o, s, \mathbf{d})$, where B is a base buffer, o is an element offset, s is the logical shape, and $\mathbf{d}$ contains element strides.

TileGrad scope	tinygrad representation	Use
Global	PARAM/BUFFER, <code>AddrSpace.GLOBAL</code>	Inputs, outputs, and invocation workspaces.
Shared	BUFFER, <code>AddrSpace.LOCAL</code>	Workgroup cooperative staging.
Private	BUFFER, <code>AddrSpace.REG</code>	Per-agent scalar or register storage.
Fragment	TileGrad logical value, lowered to REG values	Distributed tiles and matrix accumulators.

Core allocators are EMPTY, SHARED, PRIVATE, and FRAGMENT. Local shapes and layouts **MUST** be fixed during specialization.

TILE, SUBVIEW, TRANSPOSE, RESHAPE, BROADCAST, and VIEW create aliases and do not copy data. A dtype reinterpretation requires equal byte size and valid alignment.

A coordinate is storage-valid only when its mapped byte range lies within the base allocation and satisfies the required alignment. RESHAPE **MUST** be rejected unless physical-layout compatibility is proven. Views with negative strides, zero strides, or overlapping logical coordinates **MUST** be read-only unless a selected profile defines writable semantics.

A logical tile may extend beyond its base allocation. Every consuming load, store, or copy **MUST** mask, pad, or suppress invalid coordinates. Writable views with overlapping coordinates are invalid unless the operation is atomic. Core external tensors are dense and contiguous; strided external tensors are a Standard profile.

A COPY whose source and destination may overlap **MUST** either use explicit snapshot semantics, in which all logical source values are read before any destination value is written, or be rejected. It **MUST NOT** be lowered as an ordinary parallel copy when that ordering cannot be proven.

Scheduled Tile IR **MUST** retain storage scope, alignment, contiguity, layout, alias information, and bounds facts until TILECOPY and TILEMMA capability selection is complete.

Execution and Control

Form	Semantics	tinygrad axis
Kernel	A grid of unordered workgroups with fixed local geometry.	GLOBAL and LOCAL.
Serial	Ordered half-open integer iteration.	LOOP.
Parallel	Unordered logical iteration distributed by a layout.	GLOBAL or LOCAL.
Reduce	A recurrence over an associative operation.	REDUCE.
Unroll	Serial semantics with full expansion requested.	UNROLL.
Vectorized	Serial element semantics with packed execution requested.	UPCAST.

On a target without physically distinct workgroups or local agents, the pinned adapter may serialize or otherwise legally realize these logical axes. The realization **MUST** preserve Core semantics and **MUST NOT** require a Target profile.

Compile-time Python control executes during specialization. Runtime conditionals lower to predicates, selects, and gated memory operations. Core does not require runtime **while**, **break**, or **continue**.

Raw workgroup and local IDs are available to low-level builder code. Source programs **SHOULD** use logical PARALLEL and layouts instead.

Standard scheduling forms

PIPELINED has serial program semantics and may be accepted without the Scheduling profile as a scheduling hint. A claim that it emits overlapping stages requires the Scheduling profile, which versions affected storage and emits a prologue, steady state, and epilogue. A claim that it overlaps memory transactions requires the Async Pipeline profile.

PERSISTENT assigns logical work items to a fixed set of workgroups. Each item **MUST** execute exactly once in a successful invocation. Persistent scheduling is part of the Scheduling profile, not Core.

Core Tile Operations

Operation	Semantics	Portable lowering
Load	Read a coordinate; return an alternate value when masked or invalid.	INDEX+LOAD with gate.
Store	Write a coordinate only when its mask and bounds are valid.	INDEX+STORE with gate.
Copy	Transfer equal-shaped regions; pad invalid source coordinates and suppress invalid destination writes. Destination is ready on return.	Distributed LOAD/STORE plus required workgroup barriers.
Fill/Clear	Write a scalar to all valid destination coordinates; clear uses zero.	Distributed STORE.
Map/Select/Cast	Pointwise fragment and scalar operations with right-aligned broadcasting.	TINYGRAD elementwise UOps.
MMA	Accumulate logical $A \times B$ into C ; no implicit clear.	Canonical MUL+REDUCE or scalar MULACC.
Reduce	Sum, product, maximum, or minimum over declared axes and accumulator dtype.	REDUCE; minimum may use an equivalent decomposition.
Scan	Inclusive or exclusive sum or maximum over one axis.	Ordered RANGE and private recurrence.
Barrier	Synchronize the workgroup and shared-memory accesses.	BARRIER.

Copy and other cooperative operations are collective over their resolved participant layout. A participant **MUST** reach each dynamic collective instance exactly once. The compiler **MUST** reject a collective under control flow when

convergence cannot be established.

TILECOPY and TILEMMA **MUST** retain their logical identity, extents, masks, layouts, participant mapping, storage scopes, alignment, pipeline stage, and requested mechanism through scheduled Tile IR and capability selection. Legalization **MUST NOT** scalarize or flatten either operation before determining whether an advertised target-aware lowering is eligible.

After selecting a portable or profiled lowering, tile operations may be eliminated into typed TINYGRAD operations or a documented UOp pattern whose recognition and schedule preservation are tested for the pinned TINYGRAD revision. Portable COPY always has a distributed load/store lowering. A profile may select vectorized, cooperative, swizzled, or asynchronous transfer only when its capability contract admits the scopes, layout, alignment, shape, and participant geometry.

MMA **MUST** specify operand orientation, logical contraction axes, input dtypes, accumulator dtype, and result layout. Native lowering **MUST** preserve that logical contract even when the selected native atom has different internal fragment geometry.

Layouts

A layout maps a logical coordinate to a participant and one or more local slots. Core layouts are:

- row-major and column-major distributions;
- blocked and per-thread tile distributions;
- replicated distributions;
- composition by reshape, repeat, and permutation.

Layouts may apply to PARALLEL, fragments, shared storage, and tile operations. A source-specified layout is exact. When the source requests inference, the compiler may select a target- and capability-aware legal layout; after selection, the resolved layout becomes exact for the specialization and **MUST** be inspectable.

Instruction fragment layouts are owned by the pinned TINYGRAD **TensorCore** configuration. TileGrad owns the outer tile and participant schedule but **MUST NOT** duplicate the renderer's native matrix atom mapping.

Shared-memory bank swizzles and instruction-specific layouts belong to Target profiles. They **MUST NOT** appear in the Core API under instruction names.

Every performance capability **MUST** declare compatible outer layouts, fragment layouts, shared-memory layouts, alignment, and participant geometry. Legalization **MUST** either choose a compatible inferred layout or reject an incompatible explicit layout; it **MUST NOT** silently reinterpret an explicit mapping. A selected shared-memory swizzle or bank layout **MUST** survive lowering in a typed or documented representation understood by the pinned TINYGRAD adapter.

Schedule Ownership

The following source decisions are exact:

- grid and workgroup geometry;
- logical tile shapes;
- execution-axis kind and nesting order;
- storage scope and explicit storage versions;
- participant and fragment layouts;
- explicit barriers and requested mechanism **portable** or **native**.

Legalization may rewrite an exact decision only as required to represent the same mapping on the selected target. It **MUST NOT** silently replace the source schedule with a different schedule.

The following are non-semantic hints:

- vector width and unroll factor;
- pipeline depth when its mode is **auto**;
- persistent traversal order and rasterization;
- occupancy, cache, and coalescing preferences;

- mechanism selection when its mode is `auto`.

`portable` requires the Core lowering. `native` requires a compatible advertised profile, including Native Matrix or a Target profile, and **MUST** reject when unavailable. `auto` may select any eligible lowering without changing program semantics.

Autotuning may search only parameters or alternatives explicitly declared tunable by the program or selected profile. It **MUST NOT** alter an exact source decision. The search space, capability tuple, benchmark method, selected configuration, and legalized IR **MUST** be reproducible and inspectable.

A performance claim requires target-aware selection of `TILECOPY` and `TILEMMA` layouts, vectorization, staging, pipeline depth, and launch geometry. Disabling `TINYGRAD` heuristic optimization does not by itself establish that the resulting schedule is performant.

Synchronization and Effects

Source order orders operations within one agent and within one collective. `BARRIER` orders workgroup shared-memory accesses. Two unordered accesses to the same address are invalid when at least one is a non-atomic write.

The compiler **MUST** preserve read-after-write, write-after-write, write-after-read, and loop-carried dependencies before reordering operations or reusing storage. Each collective, barrier, token, and profiled memory operation **MUST** declare its participant group, affected memory scope, and ordering effects. Separate workgroups **MUST** communicate only through typed atomics or an advertised cooperative, cluster, or grid mechanism with matching launch support.

Core provides only workgroup synchronization. Subgroup, cluster, cooperative grid, and async transaction synchronization require profiles with matching typed UOps and runtime launch support. A workgroup barrier **MUST NOT** be used as a fallback for a wider scope.

Async operations return a typed token. A dependent access is invalid until a wait on that token establishes completion. This rule is normative for the Async Pipeline profile; Core has no async tokens.

An Atomics profile **MUST** define each operation's result, supported memory orders and scopes, and target mapping. Unsupported operation, order, and scope combinations **MUST** be rejected.

Lowering to TINYGRAD

Stage	Responsibility
Specialize	Bind compile-time and shape values and form the cache key.
Tile IR	Check types, shapes, views, storage, bounds, and collective legality.
Scheduled Tile IR	Resolve execution axes, layouts, storage versions, and scheduling requests.
Legalized Tile IR	Select portable or profile operations and make masks, ownership, and barriers explicit.
UOp Sink	Emit a scheduled <code>SINK</code> with <code>KernelInfo</code> through <code>Tensor.custom_kernel</code> .
Program	Delegate required legalization, rendering, compilation, launch, and execution, program caching, and JIT replay to <code>TINYGRAD</code> .

Core kernels emit `PARAM`, `BUFFER`, `RANGE`, `END`, `INDEX`, `LOAD`, `STORE`, `AFTER`, `GROUP`, `REDUCE`, `BARRIER`, `SPECIAL`, constants, casts, and elementwise UOps admitted by the pinned `TINYGRAD` specification. The integration contract **MUST** list the exact admitted operations for each pinned revision range. Equivalent renamed or decomposed typed operations may be used only when that contract documents them.

The adapter **MUST** preserve every exact schedule decision while allowing `TINYGRAD`'s required range expansion, reduction lowering, local-buffer materialization, GPU-dimension lowering, load/store insertion, dtype legalization, linearization, rendering, and compilation. Those passes may legalize representation but **MUST NOT** select a new execution schedule.

Required `TINYGRAD` legalization passes may change representation, but they **MUST** preserve the selected `TILECOPY` and `TILEMMA` participant mapping, layouts, memory scope, barriers, and capability mode. Integration tests **MUST** verify this preservation in both the UOp graph and rendered program.

An empty `KernelInfo.opts_to_apply` disables heuristic and beam optimizations but is not by itself a complete schedule lock. The pinned integration **MUST** provide and test a supported schedule-preserving mechanism; it **MUST NOT** rely on undocumented UOp tags.

A TILEGRAD 1.0 pairing is nonconforming unless its pinned integration has a documented and tested schedule-preserving custom-kernel path. Optimizer accidents, an empty option list, or a passing correctness test alone are insufficient.

Capability Profiles

A capability profile is a versioned contract among TILEGRAD, a target predicate, and a pinned TINYGRAD revision range. Each advertised capability tuple **MUST** identify:

- target, renderer, and compatible TileGrad and TINYGRAD revisions;
- operations and typed UOps or documented typed recognition patterns;
- supported dtypes, shapes, alignments, layouts, and storage scopes;
- participant geometry and required launch metadata;
- effects, synchronization, and completion semantics;
- numerical mode and inspection signature; and
- correctness gates and, when performance is claimed, benchmark gates.

Profile support **MUST** be determined from this record rather than target-name checks or untyped renderer source injection. An exact profile request **MUST NOT** silently fall back. In `auto` mode, an implementation may use the portable lowering when no eligible capability tuple exists, but inspection **MUST** report the rejected native candidates and reasons.

Native Matrix Profile

The Native Matrix profile may be advertised only for a capability tuple for which the pinned TINYGRAD integration provides a typed native matrix operation or documented typed recognition path, a renderer-advertised configuration, compatible input and accumulator dtypes and shapes, compatible layouts and participant geometry, end-to-end compilation and launch support, and inspection proving that native matrix code was emitted. TINYGRAD owns the native atom's instruction shape, lane mapping, operand swizzle, and final native matrix UOp.

One versioned integration strategy may lower MMA to a canonical multiply-reduce form and apply an explicit `OPTOPS.TC` request. A `native` claim succeeds only when post-lowering inspection confirms the corresponding typed native matrix UOp. An ignored or unsuccessful request **MUST** be treated as unavailable, not as native execution. The strategy may change without changing TileGrad language semantics.

TileGrad source **MUST NOT** construct renderer-internal native matrix arguments directly. An unavailable dtype, accumulator type, shape, or target **MUST** either use the portable MMA lowering in `auto` mode or reject in `native` mode.

Capability claims are made independently per target, TINYGRAD revision range, renderer, dtype tuple, accumulator type, admitted shape set, and layout contract. CUDA MMA, AMD MFMA/WMMA, and Metal simdgroup operations are candidate families; naming a family does not claim support for every renderer or configuration in that family.

Native Matrix targets TileLang-like throughput for ordinary dense and edge-tiled matrix workloads on advertised capability tuples. Each performance claim **MUST** pass the versioned benchmark gates for its tuple. Exact TileLang parity, parity on unsupported tuples, and parity for asynchronous or target-specific mechanisms are not implied.

Standard Profiles

Profile	Operations	Integration requirement
Vector and Cooperative	Vectorized/coalesced synchronous copy, distributed fill, map, and reduction.	Typed vector memory operations, participant layouts, and target-aware legality checks.
Scheduling	PIPELINED, PERSISTENT, rasterization.	Scheduled Tile IR storage versioning; no async behavior required.
Strided Async Pipeline	Non-contiguous external tensor views. Async copy, commit, wait, transaction barriers.	Correct UOp indexing and alias validation. Typed async UOps with explicit dependencies and completion tokens.
Atomics	Atomic load/store, exchange, compare-exchange, and RMW with scope and order.	Typed atomic UOp support in each claimed renderer.
Subgroup	Shuffle, ballot, vote, match, and subgroup reduction.	Typed subgroup UOps with declared width.
Sparse and Packed	Structured sparse MMA, packed integer and low-precision formats.	Typed native operation and metadata layout.

Profile support is determined during specialization from the capability record. Unsupported operations **MUST** report the source operation, requested profile, target, renderer, dtype, shape, layout, and missing integration feature. A Scheduling performance claim **MUST** define the variants permitted for tuning and their benchmark gates; storage versioning alone establishes correctness, not performance. Async Pipeline does not imply TMA, WGMMA, or cluster support.

Target Profiles

Target profiles expose mechanisms that are not portable language concepts, such as multidimensional hardware transfers, advanced matrix instructions, clusters, descriptor storage, target memory, register budgets, and cooperative launch.

Target operations **MUST** be namespaced, for example `T.cuda.*`, and **MUST** have typed effects, validation, renderer support, and runtime launch support in the pinned integration. Raw `CUSTOM` source strings are an experimental escape hatch and do not establish profile conformance.

Expected future target families include async copy, TMA-like transfers, WGMMA, TCGEN05, tensor memory, cluster barriers, cooperative launch, sparse matrix operations, and block-scaled formats. Listing a family does not require a 1.0 implementation to support it.

The Standard Async Pipeline profile covers target-neutral typed async semantics. Descriptor-based multidimensional transfers, proxy fences, bulk transaction accounting, and TMA-specific behavior belong to Target profiles. TMA-like transfers, WGMMA, TCGEN05, tensor memory, clusters, and cooperative launch **MUST NOT** be advertised unless the pinned TINYGRAD integration provides typed operations, effects, synchronization, and launch metadata sufficient to represent them. Otherwise they are unsupported by TILEGRAD 1.0.

Non-Normative Integration Roadmap

This section is informative and does not define language conformance. TileGrad should add capabilities by contributing small typed primitives to TINYGRAD, not by hiding major operations in source strings. A practical implementation order is:

1. a stable target-capability and resource-query API;
2. a documented native matrix selection and inspection path;
3. typed vector and shared-to-fragment memory operations;
4. generic typed atomics;
5. typed subgroup shuffle, ballot, and vote;
6. async copy with commit/wait tokens;

7. scoped barriers and async memory ordering;
8. cluster and cooperative launch metadata;
9. additional native matrix and target-storage operations as required.

These additions are integration milestones, not Core language requirements. Schedule preservation is already a 1.0 integration requirement, not a roadmap item. A profile that depends on a roadmap item **MUST NOT** be claimed until its typed integration is complete.

Inspection and Diagnostics

The implementation **MUST** expose:

- parsed Tile IR;
- scheduled and legalized Tile IR;
- selected profiles and native matrix configuration;
- capability-record version and rejected native candidates;
- TILECOPY and TILEMMA lowering choices and resolved layouts;
- tuned schedule parameters and benchmark-manifest entry;
- emitted UOp Sink and KernelInfo;
- rendered source and launch geometry when provided by TINYGRAD.

Errors **MUST** distinguish invalid source, unsupported profile, invalid UOp construction, backend compilation failure, and runtime failure.

Conformance

An implementation claiming **Core 1.0** conformance **MUST** support both frontends, specialization, Core storage and control, all Core tile operations, explicit layouts, workgroup semantics, inspection, and portable lowering on every target for which its pinned TINYGRAD integration can legally lower and execute the Core UOps required by the test program. Core conformance **MUST NOT** depend on native matrix, async, subgroup, cluster, or other optional profiles. The release **MUST** publish a matrix of compatible and execution-tested targets. A generated TILEKERNEL **MUST** execute correctly inside a precompiled TINYGRAD **@function** and an enclosing **TinyJit**.

Core workload gates are:

- masked multidimensional copy and transpose with irregular tails;
- edge-tiled mixed-precision GEMM through the portable MMA path;
- reductions and scans with irregular tails;
- numerically stable softmax and a fused LayerNorm-style kernel; and
- a fused GEMM epilogue that exercises layout and schedule preservation.

A versioned performance manifest **MUST** define, for every claimed capability tuple, benchmark workloads, shapes, dtypes, layouts, warmup, timing method, baseline and version, threshold, permitted variance, and correctness tolerance. Continuous conformance testing **MUST** reject regressions beyond the published threshold. Core correctness conformance alone **MUST NOT** be described as a performance claim.

A **Native Matrix** claim **MUST** run identical inputs, masks, and tails through portable and native paths, satisfy the pinned TINYGRAD numerical behavior, prove emission of the selected typed native matrix operation, and pass the target-specific ordinary-MMA benchmark gates. The gates **SHOULD** cover representative square, rectangular, edge-tiled, and fused-epilogue GEMMs and **MUST** include a case where target-aware TILECOPY and layout selection affect throughput.

The official TileGrad 1.0 release **MUST** satisfy Core on its published compatible-target matrix and **MUST** satisfy Native Matrix on at least one suitable typed capability tuple. Native Matrix is not required on targets without suitable support. Additional implementations may claim Core 1.0 without claiming Native Matrix.

Each additional profile claim **MUST** include positive execution tests, rejection tests for unsupported targets and shapes, inspection of the relevant typed UOps, and a pinned TINYGRAD revision range. A profile making a

performance claim **MUST** also pass its versioned benchmark gates; a profile making semantic claims only need not assert a speedup.

Implicit autograd is not required. Explicit backward kernels are ordinary TileGrad programs. An implementation may integrate `Tensor.custom_kernel`'s `grad_fxn` hook.

Non-Goals

- Exact TileLang source compatibility.
- Exact or universal TileLang performance parity across targets, dtypes, shapes, or mechanisms. Performance claims are capability-specific.
- A TVM-style target registry or a second backend compiler.
- A TileGrad runtime JIT, execution-graph capture system, or compiled program cache separate from TINYGRAD.
- Exposing every vendor instruction in the Core namespace.
- Replacing TINYGRAD tensor-core selection, rendering, runtime, or numerical semantics.
- Claiming async, TMA, WGMMMA, cluster, atomic, cooperative-launch, or target-memory semantics through untyped source injection.